CSE 4208: Computer Graphics Laboratory

Auschwitz I
Educational 3D Reconstruction

By

Raufun Ahsan
Roll: 2007030

Submitted to

Dr. Sk. Md. Masudul Ahsan
Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Md Tajmilur Rahman
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 Core Orchestration . . . 3
2.2 Zone-based World Composition . . . 3
2.3 World Scale Notes . . . 3

3 Rendering Pipeline **3**
3.1 4-Point View Mode . . . 4

4 Geometry and Modeling Details **4**
4.1 Primitive-based Construction . . . 4
4.2 Curved Surfaces . . . 4
4.3 Procedural Objects . . . 5
4.4 Flyweight-based Optimization . . . 5

5 Lighting and Time System **5**
5.1 Lighting Budget . . . 5
5.2 Dynamic Behavior . . . 5
5.3 Bounded Light Upload . . . 5
5.4 Universal Runtime Material/Lighting Toggles . . . 6

6 Feature Implementations **6**
6.1 Train System . . . 6
6.2 Soldiers Courtyard . . . 6
6.3 Interiors and Distinct Zones . . . 7

7 Texture Pipeline and Samples **7**
7.1 Texture Handling . . . 7
7.2 Texture Sample Sheet . . . 7

8 Day and Night Sky **8**

9 Scene Screenshots **9**

10 Keyboard Controls **10**

11 Conclusion **10**

12 References **10**

List of Figures

1 4-point view mode (day) showing X, Y, Z, and isometric perspectives. 4
2 Train system placed outside the entrance-side fence. 6
3 Primitive soldier modeling and parade placement. 6
4 Interior variants used across zone implementations. 7
5 Texture sample sheet (all available samples). 8
6 Sky appearance across the day-night cycle. 9
7 Selected scene captures from implemented zones and view modes. 9
8 Procedural and atmospheric features. 10

1 Introduction

This project is a real-time OpenGL 3.3 reconstruction of Auschwitz I, implemented in C++17 with a modular scene architecture. The current implementation is centered around a zone-based world model, multi-pass rendering, and runtime interaction systems.

The implementation focuses on:

- clean separation of world sectors into independent zone modules,
- reusable primitive-driven modeling (cube, cylinder, sphere, plane, pyramid),
- dynamic lighting with bounded light budgets,
- procedural geometry for trees and curved structures,
- interactive controls for scene behavior, visualization mode, and material/lighting toggles.

2 System Architecture

2.1 Core Orchestration

The entry point in `src/main.cpp` initializes GLFW/GLAD, shaders, input callbacks, and the frame loop. The central orchestrator `src/Scene.h` owns all subsystems and zones, controls texture loading/fallback, and executes the full render pipeline.

2.2 Zone-based World Composition

The world is built from dedicated modules under `src/zones/`, including:

- `GroundZone`, `BarrackGrid`, `BarrackInteriors`,
- `EntranceGate`, `FenceSystem`, `GuardTowers`, `StreetLamps`,
- `Block11Zone`, `CrematoryZone`, `AdminZone`,
- `TrainSystem` (line + locomotive + five bogeys),
- `SoldiersCourtyard` (formation grid + parade animation).

2.3 World Scale Notes

Barrack placement currently renders 24 barracks in a 4x6 grid with fixed row/column centers and consistent spacing. The entrance gate is positioned on the eastern side, while the train line is parallel to the entrance fence and outside the perimeter.

3 Rendering Pipeline

The frame is rendered in seven passes:

1. Skybox,
2. Stars,
3. Opaque geometry (Phong),
4. Celestial/unlit emissive bodies,
5. Alpha/transparency pass,
6. Particle pass,
7. HUD pass.

This organization keeps opaque and transparent content separate, supports atmosphere effects, and allows HUD rendering after scene compositing.

3.1 4-Point View Mode

A dedicated viewport mode splits the screen into four equal quadrants and renders:

- X-axis perspective,
- Y-axis perspective,
- Z-axis perspective,
- isometric perspective.

This mode is toggleable at runtime and reuses the same scene rendering path per quadrant.

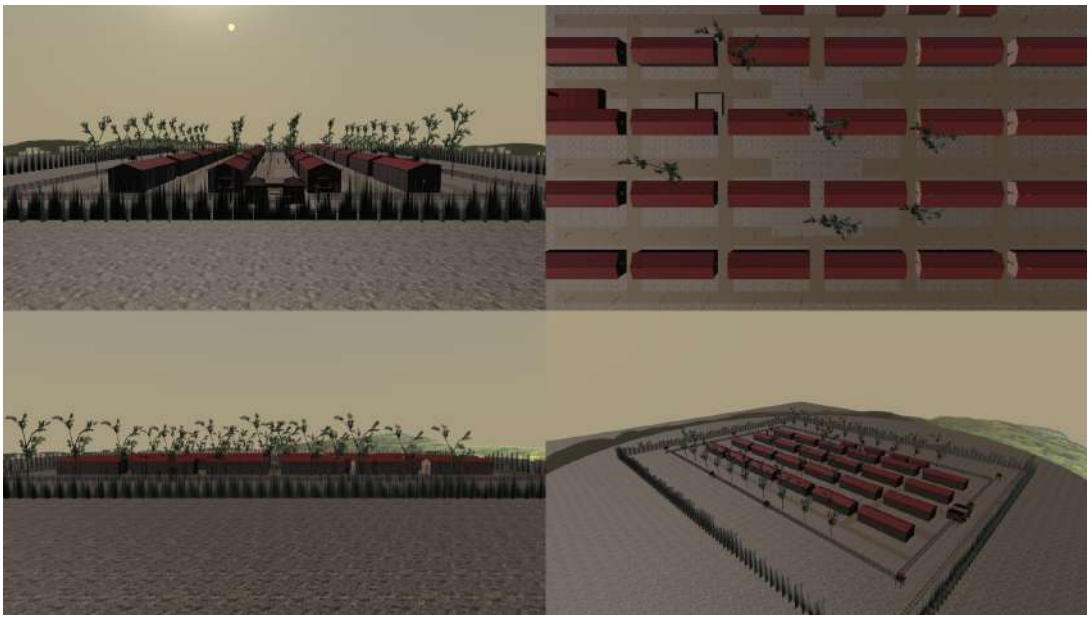


Figure 1: 4-point view mode (day) showing X, Y, Z, and isometric perspectives.

4 Geometry and Modeling Details

4.1 Primitive-based Construction

Most structures are assembled from shared primitive meshes, with transformation-based composition for walls, roofs, poles, rails, furniture, and character parts.

Primitive roles in this project:

- **Cube**: main building blocks (walls, floors, roofs, frames, furniture, soldier body parts).
- **Cylinder**: poles, columns, lamp posts, tower legs, and circular structural members.
- **Sphere**: moon/sun bodies, lamp bulbs, and rounded character features.
- **Plane**: large ground and road surfaces, courtyard pads, and horizon extensions.
- **Pyramid**: decorative roof caps and entrance-gate details.

4.2 Curved Surfaces

Curved elements use Bezier and ruled-surface generation.

Cubic Bezier centerline:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \tag{1}$$

$$\mathbf{T}(t) = \frac{d\mathbf{B}(t)/dt}{\|d\mathbf{B}(t)/dt\|}. \tag{2}$$

Tube sweep around the curve frame:

$$\mathbf{S}(t,\theta) = \mathbf{B}(t) + r\big(\cos\theta\,\mathbf{N}(t) + \sin\theta\,\mathbf{B}_f(t)\big). \tag{3}$$

Ruled surface interpolation between two boundary curves:

$$\mathbf{R}(u,v) = (1-v)\mathbf{C}_1(u) + v\mathbf{C}_2(u). \tag{4}$$

These are used for features such as curved lamp arms, gate arch components, train boiler shape, and crematory roof profiles.

4.3 Procedural Objects

Besides curved surfaces, the scene uses procedural updates for trees, stars, and particles:

L-system branch scaling (tree recursion):

$$L_{k+1} = \alpha L_k, \quad r_{k+1} = \beta r_k, \quad 0 < \alpha, \beta < 1. \tag{5}$$

Upper-hemisphere star sampling:

$$\mathbf{p}(\theta,\phi) = R\begin{bmatrix}\cos\phi\cos\theta \\ \sin\phi \\ \cos\phi\sin\theta\end{bmatrix}, \quad \theta \in [0, 2\pi],\ \phi \in \left[0, \tfrac{0.45\pi}{}\right]. \tag{6}$$

Particle position update (snow/smoke style integration):

$$\mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mathbf{v}_t\,\Delta t. \tag{7}$$

4.4 Flyweight-based Optimization

The project uses a flyweight-style mesh container (`MeshFlyweight`) for generated meshes. Shared VAO/VBO/EBO data is created once and reused for repeated draws with different transforms/material state. This reduces duplicate geometry allocation for repeated curved assets.

5 Lighting and Time System

5.1 Lighting Budget

The Phong shader path supports:

- 2 directional lights (sun + moon),
- up to 32 point lights,
- up to 36 spot lights.

5.2 Dynamic Behavior

The day-night cycle updates sun/moon direction and intensity continuously. Night behavior includes lamp activation, stars, and moving tower searchlights.

5.3 Bounded Light Upload

To keep active light counts stable:

- interior point lights are selected by distance,
- lamp spotlights are sorted by camera distance and capped,
- one spotlight slot is reserved for the train headlight.

5.4 Universal Runtime Material/Lighting Toggles

Global keyboard controls allow toggling ambient, diffuse, specular, emissive terms and all texture sampling. This supports direct visual inspection of material and light contributions.

6 Feature Implementations

6.1 Train System

The train subsystem introduces an external rail line (two metal rails + wooden sleepers) and an animated train with one engine and five bogeys. Motion state is scripted as:

1. approach from outside horizon,
2. stop near gate,
3. depart past horizon.

A front spotlight is integrated into the global spotlight upload path.

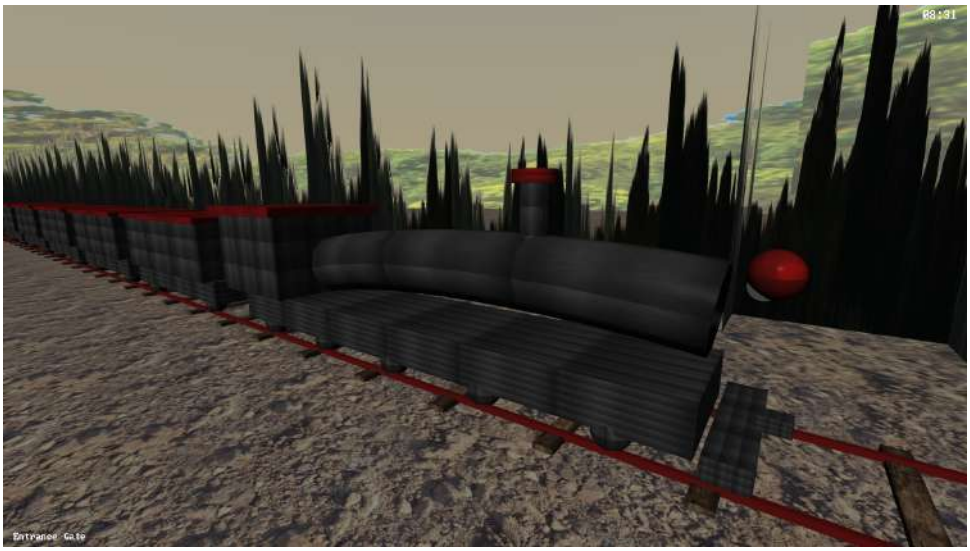
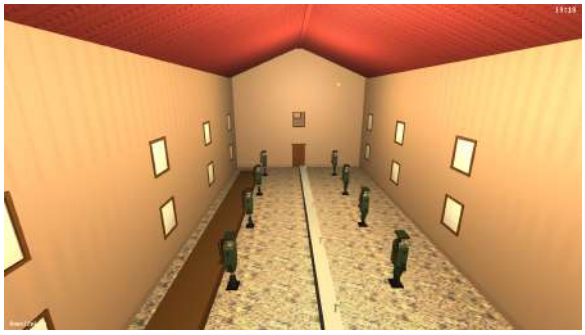


Figure 2: Train system placed outside the entrance-side fence.

6.2 Soldiers Courtyard

A central courtyard includes a grid of primitive human figures (body, head, arms, hands, legs, feet, back-mounted rifle). Parade mode animates arm swing and leg gait while maintaining planted feet behavior.

(a) Courtyard formation

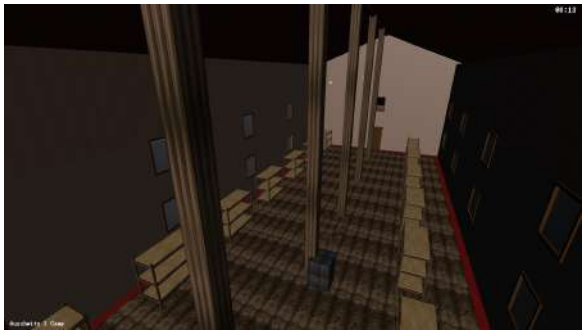


(b) Formation in enclosed space

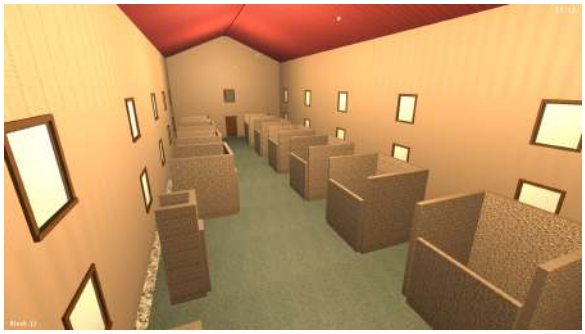
Figure 3: Primitive soldier modeling and parade placement.

6.3 Interiors and Distinct Zones

The project includes dedicated zone logic for barracks, gate, administrative area, crematory area, and special block interiors (including jail-style interior layout for Block 11).



(a) House-style interior



(b) Jail-style interior

Figure 4: Interior variants used across zone implementations.

7 Texture Pipeline and Samples

7.1 Texture Handling

Textures are loaded from relative paths and used with repeat/clamp behavior depending on target surface. If a texture is unavailable, fallback solid textures are generated to keep rendering stable.

7.2 Texture Sample Sheet

Figure 5 contains all currently provided texture samples from `assets/texture_samples/`.

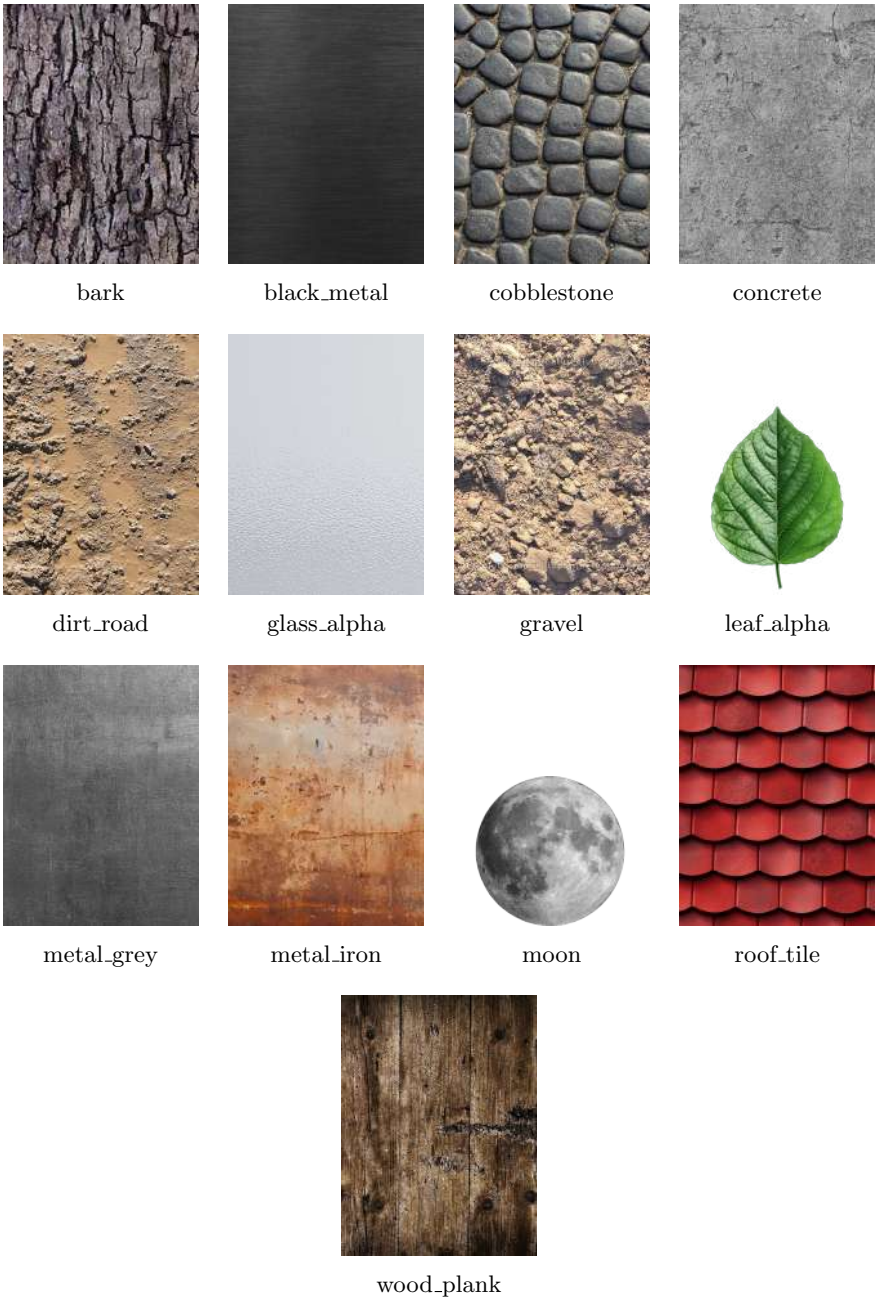


Figure 5: Texture sample sheet (all available samples).

8 Day and Night Sky

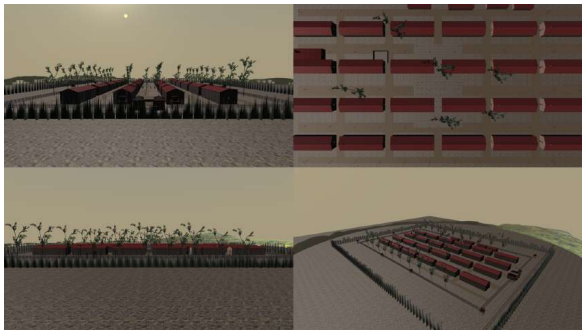
The sky rendering is driven by the day-night cycle and includes color interpolation, star visibility at night, and moon rendering in the celestial pass. The two captures below show the daytime and nighttime sky states used by the scene.

(a) Day sky state

(b) Night sky state

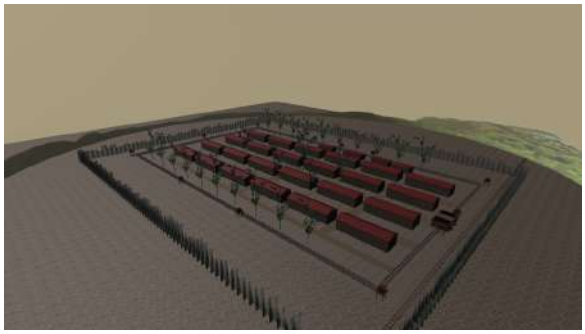
Figure 6: Sky appearance across the day-night cycle.

9 Scene Screenshots

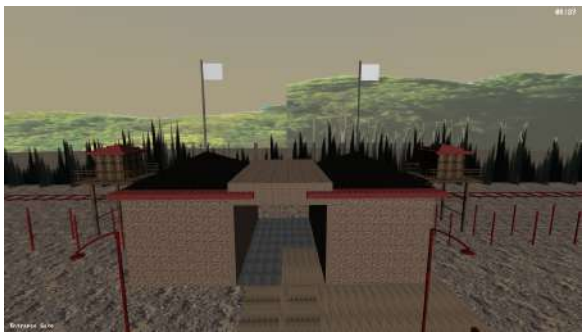


(a) Camp overview (day)

(b) Camp overview (night)



(c) Bird's-eye/isometric view



(d) Entrance gate area

Figure 7: Selected scene captures from implemented zones and view modes.

(a) Procedural L-system tree

(b) Procedural snowflake overlay

Figure 8: Procedural and atmospheric features.

10 Keyboard Controls

Table 1: Keyboard controls in the current implementation

Key	**Action**
`W A S D`	Move camera
`Shift`	Sprint
`Space / Left Ctrl`	Move camera up / down
`Mouse / Scroll`	Look / FOV zoom
`T`	Cycle time speed (pause/1x/8x/30x)
`H`	Toggle HUD
`C`	Toggle cursor lock
`F`	Toggle fullscreen
`G`	Toggle doors
`K`	Toggle windows
`J`	Trigger train run
`U / I`	Increase / decrease train speed
`P`	Toggle soldier parade
`V`	Toggle normal / 4-point view
`1 / 2 / 3 / 4`	Toggle ambient / diffuse / specular / emissive
`0`	Toggle all textures globally
`ESC`	Exit application

11 Conclusion

The implemented system is a complete, modular OpenGL reconstruction with stable multi-pass rendering, bounded dynamic lighting, procedural geometry, zone-level expansion, and runtime interaction controls. The interactive train and soldier courtyard modules along with interactive doors, windows and other controls integrate into the existing pipeline without changing the core scene architecture, demonstrating extensibility of the design.

12 References

1. Khronos Group, *OpenGL Reference Pages*. https://registry.khronos.org/OpenGL-Refpages/
2. GLFW, *GLFW 3 Documentation*. https://www.glfw.org/documentation.html
3. G-Truc, *GLM: OpenGL Mathematics*. https://github.com/g-truc/glm
4. GLAD, *OpenGL Loader Generator*. https://glad.dav1d.de/